

```
$ spring run TransactionLab --propagation=REQUIRED
```

# @Transactional



// 数据库事务 — 数据库事务

**begin**

开始

**commit**

提交 flush

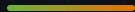
**rollback**

回滚

**proxy**

代理

\$ cat agenda.md



01 /



□□□□□□□□

02 /



□□□□□□□□

03 / AOP □□

□□□□□□□□

04 /



□□□□rollback □□□FAQ

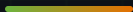
🔗🔗🔗🔗🔗🔗🔗 @Transactional □□□□

PART 01

@Transactional □□□□

□□□□□□□□□□

□□□□□□□□



1. 

□□□□

2. A□□

UPDATE accounts...

3. B□□

UPDATE accounts...

4. 

Commit

WHAT IF STEP 3 FAILS?

□□□□□□ — □□□□□□□□□□Rollback□□□□□ A□

@Transactional □□□□□□□□□□□□□□□□□□□□□□

□□□□□□□□□□□□

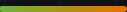
| □□   | □□                       | □□           |
|------|--------------------------|--------------|
| □□□□ | <div>EntityManager</div> | □□□□□        |
| □□□□ | □□□□□□□□□□               | □□□□□□□      |
| □□□□ | □□□□□□□□□□               | <div>→</div> |
| □□□□ | <div>flush()</div>       | <div>→</div> |

□□□□□□□□ → commit → □□ flush() □□□□□□□□ Entity □□ save()□□□□□□□□

## PART 02



□□□□□□□□□□

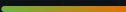


```
1 // 数据库操作
2 // 数据库操作
3 add*, save*, create*,
4 update*, delete*
```

PROPAGATION\_REQUIRED

数据库操作数据库操作

☐ RuntimeException → ☐☐

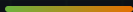


```
1 // 数据库操作
2 // 数据库操作
3 find*, get*, search*,
4 getCount*, *数据库操作
```

PROPAGATION\_NOT\_SUPPORTED

数据库操作数据库操作

setReadOnly(true)☐☐☐☐



```
1 // ✅ 测试
2 public void saveUser() { ... }
3 public void updateOrder() { ... }
4
5 // ❌ 测试@Transactional
6 public void processUser() { ... } // 测试
7 public void handleOrder() { ... } // * 测试
```

PRIORITY

测试 @Transactional 测试 process\* 测试

测试测试测试测试

CHECKPOINT\_01

□□□□□□□□□□□□□□□□

A processUser() @Transactional□□□□□□□□

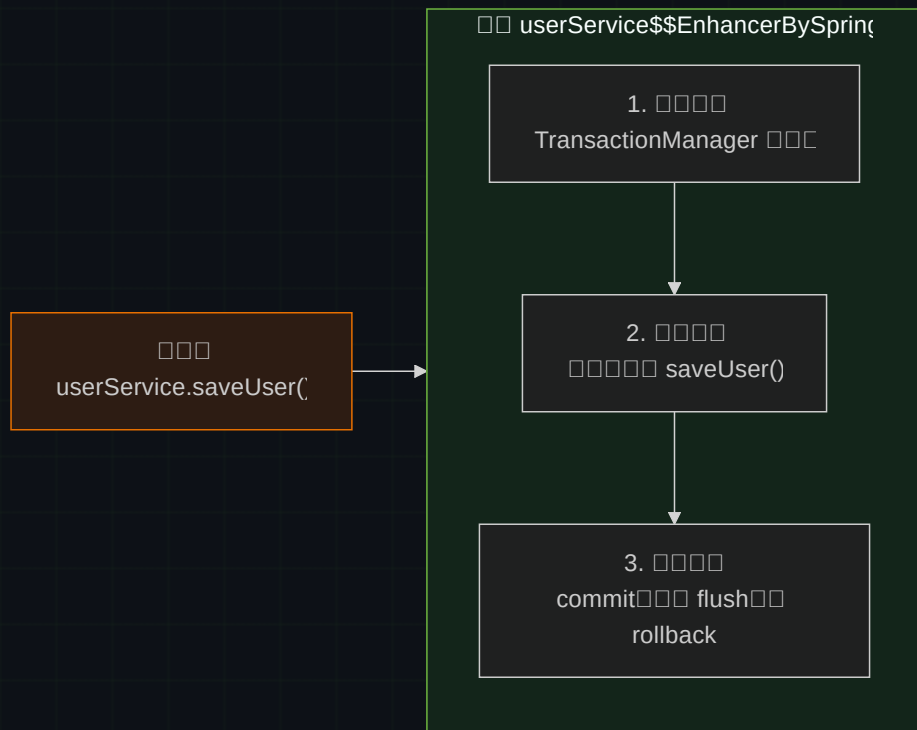
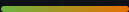
B □□□□ this.saveUser() □□□□□□□ AOP □□

C □□□□□□□□□□□□□□□□process\* □□□□□□□□

PART 03

# AOP □□□□□□□

Spring □□□□□□□□□□



## PART 04



90% ██████████



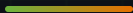
```
1  @Service
2  public class UserService {
3
4      // 🚀 快速实现
5      public void processAndSave(User user) {
6          validateUser(user);
7          saveUser(user); // this.saveUser()
8      }
9
10     public void saveUser(User user) {
11         userRepository.save(user);
12     }
13 }
```

WHY

`this.saveUser()` 需要调用 `saveUser()` 方法 --  
AOP 需要调用

1. 需要调用 `Service`

2. 需要调用 `self.saveUser()`



01 00 00 00

```
□□□□□□□process* □□□
```

02

this. ☐☐☐☐

## 03 public

```
private [ ]
```

## 04 Checked Exception

# RuntimeException

\$ ████████████████████████████████████ 90%



## DO

- ✓ 使用 save\* / update\* / delete\*
- ✓ 使用 Service 接口
- ✓ 使用 Service 接口

## DON'T

- × 使用 DAO 接口
- × 使用 DAO 接口
- × 使用 DAO 接口



□□□□□□

□□□□□commit NOT DESIGN NOT DESIGN NOT DESIGN flush NOT DESIGN NOT DESIGN rollback

□□□□□

□□□□□□□□□□□□□□

□□□□□□

this. NOT DESIGN NOT DESIGN NOT DESIGN private □□□□□

□□□□

NOT DESIGN NOT DESIGN NOT DESIGN find\* / get\* NOT DESIGN NOT DESIGN NOT DESIGN dirty checking

□□□□□□□□ — □□□□□□□□

# Transaction committed.

[[[JPA flush dirty checking ]]

```
$ spring run TransactionLab --status
```

```
begin → invoke → commit(flush) | rollback
```